

Acceso a datos en spring framework

M6: Desarrollo de aplicaciones jee con spring framework

|AE3: Implementar capa de acceso a datos en una aplicación web utilizando Spring Framework para dar solución a un problema

Introducción

La capa de Acceso a Datos se refiere a la gestión, almacenamiento y recuperación de la base de datos, permitiendo vincular los datos relacionales al lenguaje de programación de java.

En esta lección aprenderemos algunas prácticas para acceder a datos en Spring utilizando **JdbcTemplate** y **JPA**. Aprenderás a configurar un DataSource en Spring y utilizar JdbcTemplate para realizar operaciones de consulta y modificación de datos..

Las clases de entidad en JPA, representan los objetos que se almacenarán en la base de datos, por lo cuál vas a aprovechar sus capacidades para persistir objetos. Además empezaremos a darle forma a la arquitectura de cada proyecto a través de la programación en capas, donde practicarás cómo invocar un repositorio desde un servicio para obtener acceso a los datos de manera estructurada. Aprenderás sobre el uso de clases de repositorio, la realización de consultas parametrizadas y el manejo de la transaccionalidad en los servicios.

Aprendizaje esperado

Al finalizar la lección podrás:

- Implementar la capa de acceso a datos en una aplicación web utilizando Spring Framework para dar solución a un problema.

Acceso a datos mediante JDBC Template

La clase **JdbcTemplate** ejecuta consultas SQL, itera sobre el **ResultSet** y recupera los valores llamados, actualiza las instrucciones y las llamadas a procedimientos, "captura" las excepciones y las traduce a las excepciones definidas en el paquete **org.springframework.dao**.

Cuando se usa **JdbcTemplate**, la mayoría de las veces se configura en el archivo de configuración de Spring. Después de eso, se implementa usando la anotación **bean** en clases DAO.

Configurando un DataSource en Spring

Antes de utilizar **JdbcTemplate**, debes configurar un **DataSource** en tu aplicación Spring. Un **DataSource** proporciona la conexión a la base de datos. Puedes configurarlo en el archivo de configuración de Spring, ya sea utilizando el XML o las anotaciones preestablecidas. Configurar un **DataSource** en Spring es esencial para permitir el acceso a la base de datos desde tu aplicación.

Veamos la configuración del **DataSource** tanto en el XML como en la clase con anotaciones:

- 1. Configuración XML:** Si estás utilizando una configuración XML en tu aplicación, puedes configurar un **DataSource** mediante elementos **<bean>** en el archivo de configuración de Spring. Por ejemplo:

```
<bean id="dataSource"
class="org.springframework.jdbc.datasource.DriverManagerDataSource">

  <property name="driverClassName" value="com.mysql.jdbc.Driver" />

  <property name="url" value="jdbc:mysql://localhost:3306/mydatabase" />

  <property name="username" value="username" />

  <property name="password" value="password" />

</bean>
```

En este ejemplo, se utiliza **DriverManagerDataSource** como implementación de **DataSource**, y se configuran las propiedades del controlador, URL de la base de datos, nombre de usuario y contraseña.

2. **Configuración mediante anotaciones:** Si deseas utilizar la configuración basada en anotaciones, puedes utilizar la anotación **@Configuration** junto con métodos anotados con **@Bean** para definir un **DataSource**. Por ejemplo:

@Configuration

```
public class DataSourceConfig {

    @Bean //a nivel de método

    public DataSource dataSource() {

        DriverManagerDataSource dataSource = new DriverManagerDataSource();

        dataSource.setDriverClassName("com.mysql.jdbc.Driver");

        dataSource.setUrl("jdbc:mysql://localhost:3306/mydatabase");

        dataSource.setUsername("username");

        dataSource.setPassword("password");

        return dataSource;

    }

}
```

En este ejemplo, se define un método **dataSource()** anotado con **@Bean** que crea una instancia de **DriverManagerDataSource** y configura las propiedades necesarias.

Una vez que hayas configurado el **DataSource**, puedes inyectarlo en tus clases de servicio o repositorios utilizando la anotación **@Autowired**.

Inyección de JdbcTemplate

Una vez configurado el **DataSource**, ya estás en condiciones de inyectar **JdbcTemplate** en tu clase de servicio o repositorio utilizando la anotación **@Autowired**. Por ejemplo:

@Repository

```
public class UserRepository {

    private JdbcTemplate jdbcTemplate;

    @Autowired

    public UserRepository(DataSource dataSource) {

        this.jdbcTemplate = new JdbcTemplate(dataSource);

    }

    // Métodos para acceder a datos utilizando JdbcTemplate

}
```

En este ejemplo, el DataSource se inyecta en el constructor del repositorio y se utiliza para crear una instancia de JdbcTemplate.

Como podemos ver, a nivel clase en el primer renglón encontramos la anotación @Repository, esta anotación indica a Spring que la clase es un componente de tipo repositorio, es decir, que trabajará en relación directa con la base de datos.

Operaciones de acceso a datos

Una vez que hayas inyectado JdbcTemplate, puedes utilizarlo para realizar operaciones de acceso a datos, como consultas y actualizaciones. JdbcTemplate proporciona métodos preestablecidos para ejecutar consultas SQL y actualizar registros. Veamos un ejemplo:

@Repository

```
public class UserRepository {

    private JdbcTemplate jdbcTemplate;

    public List<User> getAllUsers() { //consulta a DB

        String sql = "SELECT * FROM users";

        return jdbcTemplate.query(sql, new UserRowMapper());

    }

    public void addUser(User user) { //actualización de DB

        String sql = "INSERT INTO users (name, email) VALUES (?, ?)";
```

```

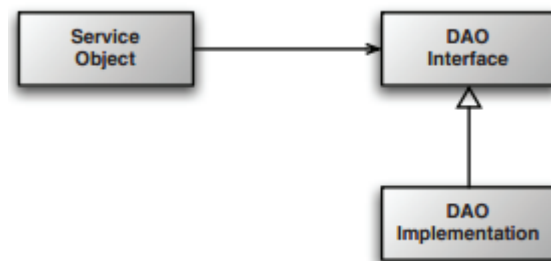
    jdbcTemplate.update(sql, user.getName(), user.getEmail());
}
}

```

En este ejemplo, `jdbcTemplate.query()` se utiliza para ejecutar una consulta y `jdbcTemplate.update()` se utiliza para realizar una actualización en la base de datos.

Creando un DAO que utiliza JdbcTemplate

"Data Access Object" es un patrón de diseño muy común que se usa para diseñar cómo los objetos de negocio de una aplicación deben usar los objetos encargados del acceso a los datos.



Los objetos del negocio acceden a los DAO a través de sus interfaces, de esta manera no dependen de una implementación en específico lo que hace que esta implementación se pueda cambiar fácilmente. También facilita las pruebas que se pueden realizar sobre esta implementación, ya que se pueden crear implementaciones falsas menos complejas que las implementaciones reales con fines de prueba.

En esta oportunidad utilizaremos MYSQL como motor de base de datos en donde vamos a crear una base de datos llamada **estudiantes**, y a continuación crearemos una tabla estudiantes con la siguiente estructura.

```

CREATE TABLE Estudiante(
  ID INT NOT NULL AUTO_INCREMENT,
  NOMBRE VARCHAR(20) NOT NULL,
  EDAD INT NOT NULL,
  PRIMARY KEY (ID)
);

```

Vamos a incluir las dependencias Maven necesarias para implementar JDBCTemplate:

```
<dependencies>

  <dependency>

    <groupId>org.springframework</groupId>

    <artifactId>spring-core</artifactId>

    <version>${spring.version}</version>

  </dependency>

  <dependency>

    <groupId>org.springframework</groupId>

    <artifactId>spring-beans</artifactId>

    <version>${spring.version}</version>

  </dependency>

  <dependency>

    <groupId>org.springframework</groupId>

    <artifactId>spring-jdbc</artifactId>

    <version>${spring.version}</version>

  </dependency>

  <dependency>

    <groupId>mysql</groupId>

    <artifactId>mysql-connector-java</artifactId>

    <version>5.1.26</version>

  </dependency>

  <dependency>

    <groupId>org.springframework</groupId>
```

```

<artifactId>spring-tx</artifactId>

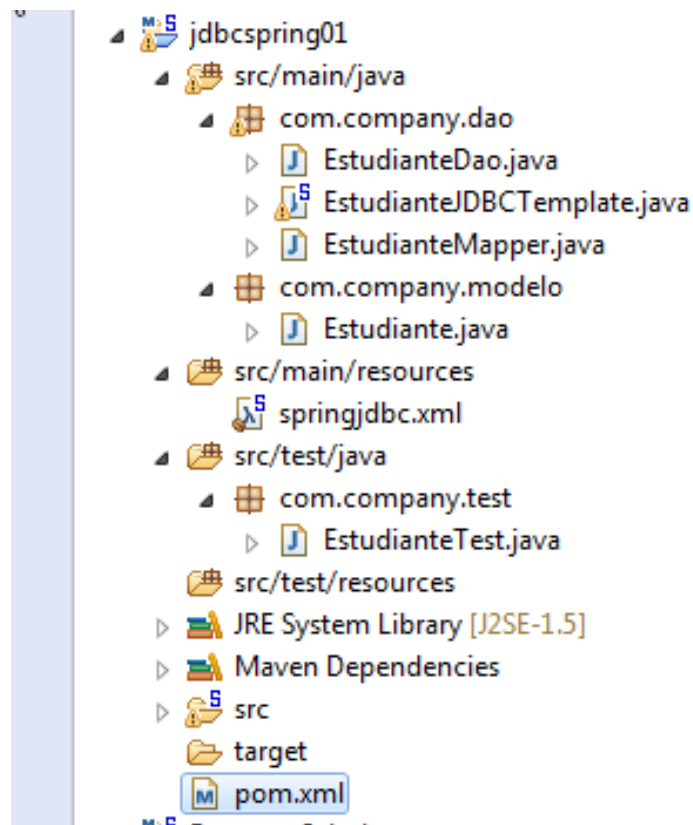
<version>${spring.version}</version>

</dependency>

</dependencies>

```

El árbol de nuestro proyecto debería quedar de la siguiente manera:



Dentro del paquete “**Modelo**”, encontraremos la clase **com.company.modelo.Estudiante.java**. Ésta clase será nuestro pojo que por la simplicidad del ejemplo sólo tendrá los campos asociados a la tabla sql con la que trabajaremos.

```

public class Estudiante {

    private Integer edad;

    private String nombre;

    private Integer id;

    // constructores - getters - setters

}

```


Podemos observar que solo tiene 3 campos **ID, NOMBRE, EDAD** que son casualmente **los campos de nuestra tabla estudiantes en MYSQL**.

Por otro lado, vamos a crear una **interfaz** DAO para esta clase Estudiante. La interfaz `EstudianteDao` se encuentra en el paquete **“dao”**, `com.company.dao.EstudianteDao.java`: aquí definimos los métodos que deberán incluir las clases que implementen esta interfaz.

```
public interface EstudianteDao {

    // Método utilizado para inicializar la conexión con la bd

    public void setDataSource(dataSource ds);

    public void Agregar(String nombre, Integer edad);

    public Estudiante getEstudiante(Integer id);

    public List listarEstudiantes();

    public void Eliminar(Integer id);

    public void Actualizar(Integer id, Integer edad);

}
```

El método `setdatasource` es el método a través del cuál podremos suministrar el datasource de conexión con la base de datos y luego la definición de los métodos asociados a un CRUD.

Realizando queries que reciben parámetros

En la clase `com.company.dao.EstudianteJDBCTemplate.java` generaremos la implementación dao, que lógicamente implementa el código para las operaciones CRUD sobre la tabla estudiante.

```
public class EstudianteJDBCTemplate implements EstudianteDao {

    private DataSource dataSource;

    private JdbcTemplate jdbcTemplateObject;

    public void setDataSource(DataSource dataSource) {

        this.dataSource = dataSource;

    }
```

```
this.jdbcTemplateObject = new JdbcTemplate(dataSource);

}

public void Agregar(String nombre, Integer edad) {

    String SQL = "insert into estudiante (nombre, edad) values (?, ?)";

    jdbcTemplateObject.update( SQL, nombre,edad);

    System.out.println(

    "Registro creado nombre= " + nombre + " edad = "  + edad);

}

public Estudiante getEstudiante(Integer id) {

    String SQL = "select * from estudiante where id = ?";

    Estudiante student = jdbcTemplateObject.queryForObject(SQL,

        new Object[]{id}, new EstudianteMapper());

    return student;

}

public List listarEstudiantes() {

    String SQL = "select * from estudiante";

    List students = jdbcTemplateObject.query(SQL,

        new EstudianteMapper());

    return students;

}

public void Eliminar(Integer id) {

    String SQL = "delete from estudiante where id = ?";

    jdbcTemplateObject.update(SQL, id);

    System.out.println("Registro eliminado ID = " + id );

    return;

}
```

```

public void Actualizar(Integer id, Integer edad) {

    String SQL = "update estudiante set edad = ? where id = ?";

    jdbcTemplateObject.update(SQL, edad, id);

    System.out.println("registro actualizado ID = " + id );

    return;

}

}

```

El código de la implementación es el que nos permite indagar sobre las ventajas de spring sobre el manejo de JDBC.

Primero lo que debemos recalcar es la utilización de 2 variables privadas:

La primera hace referencia al **datasource** de la base de datos que será sostenida en la tabla `private DataSource dataSource;`

La segunda hace referencia a la template para trabajar con JDBC que nos proporciona spring: `private JdbcTemplate jdbcTemplateObject;`

El primer método que vamos a encontrar es el `setDataSource` que como parámetro de ingreso recibe un datasource que lógicamente será suministrado por inyección dado que este objeto está precargado en el spring **bean** de la configuración.

En su interior el método asigna el valor a la variable privada para este fin, pero por otro lado genera una instancia de la template que nos proporciona Spring pasándole la conexión con la base de datos para que se encargue de ahora en más.

Modificando datos con JdbcTemplate

Los métodos que realizan operaciones de actualización en la clase **Agregar, Eliminar y Actualizar** tienen una misma lógica de funcionamiento.

Primero definimos la cadena con la sentencia SQL indicando con ? la necesidad de un parámetro externo, ejemplo:

`insert` into estudiante (nombre, edad) values (?, ?)

`delete` from estudiante where id = ?

`update` estudiante set edad = ? where id = ?

Spring se encarga de todo solo haciendo referencia al método `update` de la `template`, pasando la query y los parámetros que tiene que reemplazar en

Luego podemos analizar los métodos de buscar un registro en particular el método **`getEstudiante`** y **`listarEstudiantes()`** donde la única diferencia es que el primero retorna solo un objeto y el segundo retorna una lista de objetos, pero en su código no presentan casi ninguna diferencia dado que ambas realizan un procesamiento similar.

Para la búsqueda de un registro utiliza el método **`queryForObject`** a quien le pasamos la sentencia de consulta SQL, le pasamos la generación de la instancia para el objeto de retorno y se genera también una instancia de la clase **`Mapper`** quien recibirá el resultado de la consulta y generará un objeto para cada fila del **`ResultSet`** que reciba.

```
String SQL = "select * from estudiante where id = ?";
```

Estudiante student =

```
jdbcTemplateObject.queryForObject(
    SQL,new Object[]{id}, new EstudianteMapper());
```

Para la búsqueda de varios registros utiliza el método **`query`**, a quien le pasa la sentencia SQL y una nueva instancia del objeto mapper..

```
String SQL = "select * from estudiante";
```

List students =

```
jdbcTemplateObject.query(SQL,new EstudianteMapper());
```

Mapeando los resultados de una consulta

En nuestro ejemplo, la clase **`com.company.dao.EstudianteMapper.java`** es la encargada de realizar el mapeo de cada fila de un **`resultset`** en un **`objeto`** del tipo estudiante identificado.

```
public class EstudianteMapper implements RowMapper {
```

```

//método

public Estudiante mapRow(ResultSet rs, int arg1) throws SQLException {

    Estudiante student = new Estudiante();

    student.setId(rs.getInt("id"));

    student.setNombre(rs.getString("nombre"));

    student.setEdad(rs.getInt("edad"));

    return student;

}
}

```

Invocando un DAO desde un servicio

Para invocar un dao desde la clase de servicio correspondiente, se puede inyectar la interfaz DAO utilizando la anotación `@Autowired`, y de ésta manera acceder a los datos en los métodos de servicio. Por ejemplo:

`@Service`

```

public class EstudianteServicio {

    private EstudianteDao estudianteDao;

    @Autowired

    public EstudianteServicio(EstudianteDao estudianteDao) {

        this.estudianteDao = estudianteDao;

    }

    // búsqueda por ID

    public Estudiante getEstudianteById(int id) {

        return estudianteDao.findById(id);

    }
}

```

```
// listado de todos los estudiantes

public List<Estudiante> getAllEstudiantes() {

    return estudianteDao.findAll();

}

// guardar/crear estudiante

public void saveEstudiante(Estudiante estudiante) {

    estudianteDao.save(estudiante);

}

}
```

Acceso a datos mediante JPA

La API de Persistencia de Java (JPA)

JPA (Java Persistence API) es la propuesta estándar que ofrece Java para implementar un Framework **Object Relational Mapping** (ORM), que permite interactuar con la base de datos por medio de objetos, de esta forma. JPA es el encargado de convertir los objetos Java en instrucciones para el Manejador de Base de Datos (DBMS). El objetivo que persigue el diseño de esta API es no perder las ventajas de la orientación a objetos al interactuar con una base de datos (siguiendo el patrón de mapeo objeto-relacional)

JPA representa una **simplificación del modelo de programación de persistencia**. La especificación JPA define explícitamente la correlación relacional de objetos, en lugar de basarse en implementaciones de correlación específicas del proveedor. JPA crea un estándar para la importante tarea de la correlación relacional de objetos mediante la utilización de **anotaciones o XML** para relacionar objetos con una o más tablas de una base de datos.

Para simplificar aún más el modelo de programación de persistencia:

- La API **EntityManager** puede actualizar, recuperar, eliminar o aplicar la persistencia de objetos de una base de datos.
- JPA proporciona un lenguaje de consulta, que amplía el lenguaje de consulta EJB independiente, conocido también como **JPQL**, el cual puede utilizar para

recuperar objetos sin grabar consultas SQL específicas en la base de datos con la que está trabajando.

- El programador **no necesita programar código JDBC** ni consultas SQL.
- El entorno realiza la conversión entre tipos Java y tipos SQL.
- El entorno crea y ejecuta las consultas SQL necesarias.

La arquitectura de JPA está diseñada para gestionar **Entidades** y las relaciones que hay entre ellas.

Clases de entidad

La entidad es una clase Java simple que representa una fila en una tabla de base de datos con su formato más sencillo. Los objetos de entidades pueden ser clases concretas o clases abstractas. Podemos decir que **cada Entidad corresponderá con una tabla de nuestra Base de Datos**.

Las clases entidad se identifican con la anotación **@Entity** (a nivel clase)

El Entity Manager en JPA

EntityManager es una interfaz que gestiona las operaciones de persistencia de las entidades, ya sea crear, editar, eliminar, traer de la base de datos una entidad, etc. Es la base de todo proyecto de JPA. A su vez trabaja como factoría de las Querys.

Veamos algunos componentes de la arquitectura JPA que interactúan con el EntityManager:

EntityManagerFactory: Es una factoría de EntityManager. Se encarga de crear y gestionar múltiples instancias de EntityManager.

Persistence: Clase con métodos estáticos que nos permiten obtener instancias de EntityManagerFactory.

Query: Es una interfaz para obtener la relación de objetos que cumplen un criterio determinado.

EntityTransaction: Agrupa las operaciones realizadas sobre un EntityManager en una única transacción de Base de Datos.

Mapeo con anotaciones de JPA

Como ya hemos visto anteriormente, las bases de datos relacionales almacenan la información mediante tablas, filas, y columnas, de manera que para almacenar un objeto en la base de datos, hay que realizar **una correlación entre el sistema orientado a objetos de Java y el sistema relacional de nuestra base de datos**. JPA nos permite realizar dicha correlación de forma sencilla, realizando por sí misma toda la conversión entre nuestros objetos y las tablas de una base de datos.

Esta conversión se denomina **ORM** (Object Relational Mapping - Mapeo Relacional de Objetos), y puede configurarse a través de metadatos (anotaciones). A estos objetos, los cuales son clases comunes y corrientes, los llamaremos desde ahora **entidades**.

Las anotaciones nos permiten configurar el mapeo de una entidad dentro del mismo archivo donde se declara la clase, de este modo, relaciona las clases contra las tablas y los atributos contra las columnas. Mediante las anotaciones vamos a explicarle al ORM como debe transformar la entidad en una tabla de base de datos.

Las anotaciones comienzan con el símbolo “@” seguido de un identificador. Las anotaciones son utilizadas antes de la declaración de clase, propiedad o método. A continuación, se detallan las principales:

@Entity: Declara la clase como una Entidad.

@Table: Declara el nombre de la Tabla con la que se mapea la Entidad.

@Id: Declara un atributo como la clave primaria de la Tabla.

@GeneratedValue: Declara la manera en la que el atributo clave primaria va a ser inicializado. Esto puede ser de forma manual, automática o a partir de una secuencia determinada. La estrategia **IDENTITY** es la más simple de usar, ya que JPA asume que ésta columna es auto generada. Esto provoca que el contador de la columna incremente en 1 cada vez que un nuevo objeto es insertado.

@Column: Declara que un atributo se mapea con una columna de la tabla.

@Enumerated: Declara que un atributo es de alguno de los valores definidos en un **Enum** (lista de valores constantes). Los valores de un tipo enumerado tienen asociado implícitamente un tipo ordinal.

@Temporal: Declara que se está tratando de un atributo que va a trabajar con fechas, entre paréntesis, debemos especificarle qué estilo de fecha va a manejar en la base de datos:

- @Temporal(TemporalType.DATE)
- @Temporal(TemporalType.TIME)
- @Temporal(TemporalType.TIMESTAMP)

De forma ejemplar, podemos redefinir la clase entidad Estudiante con la que venimos trabajando en JDBC a modo de ejemplo:

@Entity

```
public class Estudiante {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Integer id;

    private Integer edad;

    private String nombre;

    // constructores - getters - setters

}
```

Capa de acceso a datos: Clases de Repositorio

En Spring un **Repository** es el componente encargado de resolver el acceso a los datos de nuestro micro-servicio. Si necesitamos guardar, modificar, eliminar registros de un **User** del sistema; será entonces el componente **UserRepository** el encargado de realizar cambios directos sobre los registros de Usuario.

Spring nos provee las funcionalidades básicas para guardar, eliminar y buscar entidades. Para esto tenemos todas las interfaces que extienden de “**org.springframework.data.repository.Repository**”. Para poder hacer uso de esto, es importante tener la dependencia de spring-data-jpa en nuestro archivo pom.xml.

```
<dependency>
```

```

<groupId>org.springframework.data</groupId>

<artifactId>spring-data-jpa</artifactId>

</dependency>

```

También es importante recordar la configuración del proyecto en el documento application.properties:

```
# MySQL DB properties
```

```
jdbc.driver=com.mysql.jdbc.Driver
```

```
jdbc.url=jdbc:mysql://localhost:3306/nombreDeLaBaseDeDatos
```

```
jdbc.username= username //(de la DB)
```

```
jdbc.password= password //(de la DB)
```

Para este ejemplo, crearemos una entidad **User** con la cual realizaremos las operaciones básicas para crear, modificar y buscar (CRUD).

CRUD es un acrónimo que refiere a las cuatro funciones mínimas sobre una entidad → **Create, Read, Update, Delete**.

@Entity

```

public class User {

    @Id @GeneratedValue(strategy = GenerationType.AUTO)

    private Long id;

    private String name;

    private String surname;

    // constructores, getters, setters

}

```

Una vez declarada nuestra clase Entidad, vamos a crear una interface **UserRepository** que **extenderá de CrudRepository**. Anotamos la clase 'UserRepository' como un componente @Repository .

CrudRepository es una interfaz genérica que recibe dos tipos. El primero es la clase que esta interfaz maneja y el segundo es el tipo de dato del ID de la entidad.

@Repository

```
public interface UserRepository extends CrudRepository<User, Long> {  
}
```

Simplemente con crear la interfaz `UserRepository` y hacerla extender de `CrudRepository` ya podemos hacer uso de diversos métodos que se encuentran previamente declarados de manera automática (sin necesidad de escribir más código) .

Los métodos que nos provee `CrudRepository` son:

- **save**: guarda una entidad.
- **saveAll**: guarda las entidades de una lista iterable.
- **findById**: busca por el identificador.
- **existsById**: verifica si existe un identificador.
- **findAll**: devuelve todos los elementos para la entidad.
- **findAllById**: busca todos los elementos que tengan el identificador.
- **count**: devuelve el total de registros de la entidad.
- **deleteById**: elimina un registro según el identificador.
- **delete**: elimina la entidad.
- **deleteAllById**: elimina todos los elementos que correspondan con el identificador.
- **deleteAll(Iterable)**: elimina todos los elementos que se reciban en el parámetro.
- **deleteAll**: elimina todos los elementos.

Recomendamos Investigar [el siguiente link](#) donde podrán encontrar las características de cada método según la documentación oficial de Spring.

Si los métodos anteriores no nos sirven para realizar consultas específicas, podemos declarar métodos de búsqueda propios en la interfaz. Utilizando la convención de nomenclatura **jpa** podremos realizar el siguiente tipo de consultas:

@Repository

```
public interface UserRepository extends CrudRepository<User, Long> {

    @Query("SELECT u FROM User u WHERE u.name = :name")

    public User buscarPorNombre(@Param("name"), String name);

}
```

En esta `@Query` personalizada, podemos realizar una búsqueda a través de los parámetros señalados. La anotación `@Query` es necesaria para que Spring sepa que tiene que evaluarlo como una consulta a base de datos.

El parámetro `":name"` hace referencia al señalado en la anotación `@Param("nombre")`, seguido por el tipo de dato a evaluar.

Relaciones entre Clases: Asociaciones

JPA nos brinda anotaciones para definir una relación entre dos clases que deseamos reflejar en la base de datos. Estas anotaciones sirven para informarle a la base de datos cuál será el tipo de relación que tendrán las tablas entre sí. Recordemos que las anotaciones cumplen el propósito de “traducir” nuestro código de Java para que lo entienda la base de datos.

Las anotaciones de asociación que nos provee JPA son:

- `@OneToOne`: relación entre tablas **uno a uno**.
- `@OneToMany`: relación entre tablas **uno a muchos**.

La relación **One-to-One (uno a uno)** en JPA establece una asociación entre dos entidades donde una instancia de una entidad se relaciona con exactamente una instancia de otra entidad. En la **entidad principal**, se utiliza la anotación `@OneToOne` junto con `@JoinColumn` para especificar la columna que almacena la clave externa.

@Entity

```
public class User {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    private String name;
```

`@OneToOne`

`@JoinColumn(name = "address_id")`

`private Address address;`

`// constructores, getters y setters`

En la entidad asociada, se utiliza `@OneToOne(mappedBy = "nombrePropiedad")` para establecer la relación bidireccional. Esta relación permite acceder a la entidad asociada desde la entidad principal y viceversa.

`@Entity`

`public class Address {`

`@Id`

`@GeneratedValue(strategy = GenerationType.IDENTITY)`

`private Long id;`

`private String street;`

`private String city;`

`@OneToOne(mappedBy = "address")`

`private User user;`

`// constructores, getters y setters`

`}`

De esta manera, se crea una relación One-to-One entre las entidades `User` y `Address`, donde cada usuario tiene una única dirección asociada y cada dirección está vinculada a un solo usuario.

La relación **One-to-Many** en JPA establece una asociación entre dos entidades **donde una entidad principal tiene una colección de entidades asociadas**. En la entidad principal, se utiliza la anotación `@OneToMany` para definir la relación, y en la entidad asociada no es necesario determinar ninguna anotación para establecer la relación con la entidad principal.

En este ejemplo, la entidad `Department` es la entidad principal y tiene una **colección de `Employee`** utilizando la anotación `@OneToMany`. La anotación `@JoinColumn` especifica la columna en la tabla `Employee` que se utilizará para almacenar la clave externa que se vincula con la tabla `Department`.

@Entity

```
public class Department {

    @Id

    private Long id;

    // otros atributos

    @OneToMany

    @JoinColumn(name = "department_id")

    private List<Employee> employees;

    // constructores, getters y setters

}
```

Por otro lado, la entidad **Employee** no tiene una referencia directa a la entidad **Department**. La relación se establece a través de la columna **department_id** en la tabla **Employee**.

De esta manera, se crea una relación **One-to-Many** donde un departamento puede tener múltiples empleados. Cada empleado está asociado a un departamento específico, pero no tiene conocimiento directo de su departamento.

Invocando un Repositorio desde la clase de Servicio

Al invocar el repositorio desde el servicio puedes acceder a todas las operaciones definidas en el repositorio, como consultas personalizadas, operaciones de persistencia, eliminación, etc. Esto te permite encapsular la lógica de acceso a datos en el repositorio y utilizarla de manera conveniente desde el servicio.

Veamos un ejemplo:

@Service

```
public class CursoService{

    @Autowired
```

```
private CursoRepository cursoRepository;  
  
public List<Curso> listAllCursos() {  
    return cursoRepository.findAll();  
}  
  
public Curso addCurso(Curso curso) {  
    return cursoRepository.save(curso);  
}  
  
public int removeCurso(int id) {  
    cursoRepository.delete(id);  
  
    return 1;  
}  
  
public Curso updateCurso(Curso curso) {  
    return cursoRepository.save(curso);  
}  
}
```

Manejo de la transaccionalidad en los servicios

¿Qué es una transacción y por qué es importante?

Una **transacción** de base de datos es un conjunto de instrucciones que se ejecutan en bloque. Por ejemplo, hacemos una consulta, modificamos un registro A en la base de datos y eliminamos un registro B. Si en alguna de estas instrucciones se produce un error todo el proceso se echa atrás. De esta manera, si luego consultamos la base de datos podremos ver que el registro A no ha sido alterado.

A este proceso de “tirar atrás” las instrucciones realizadas se le dice hacer un **rollback**, mientras que el proceso de confirmar todas las instrucciones en bloque

una vez hemos visto que no se ha producido ningún error se le llama hacer un **commit**.

Una transacción es descrita tradicionalmente en desarrollo de software por medio de cuatro factores (**ACID**)

Atómico: Las transacciones están compuestas de una o más actividades agrupadas como una sola unidad de trabajo. Este factor asegura que todas las operaciones en una transacción ocurrirá **si todas las operaciones terminan su ejecución correctamente** o ninguna de ellas ocurrirá si cualquiera de estas actividades termina su ejecución incorrectamente, en cuyo caso se deshacen todos los cambios efectuados por las operaciones de la transacción.

Consistente: Una vez que una transacción termina su ejecución, ya sea correcta o incorrectamente, el sistema es dejado en un estado consistente con el negocio que modela.

Aislado: Las transacciones deben permitir a varios usuarios trabajar con los mismos datos, sin que cada trabajo de un usuarios se enrede con el de los demás. Por lo tanto, las transacciones **deben ser aisladas unas de las otras**, previendo de esta manera la lectura y/o escritura concurrente de los datos.

Durable: Una vez que una transacción concluye, el resultado de una transacción debe **persistir** y debe sobrevivir a cualquier tipo de fallo del sistema.

Configurando la transaccionalidad

La manera más común de tratar las transacciones en Spring es mediante la anotación **@Transactional** en la cabecera del método de una clase (nunca un interfaz) gestionada por Spring:

@Transactional

```
public void hazAlgoTransaccionalmente() {  
    // Soy transaccional!  
}
```

La regla es simple, la anotación **sólo tiene efecto en la cabecera de un método** por el cual se acceda a la clase. Evidentemente se puede marcar todos los métodos como transaccionales, e incluso la propia clase (con lo que todos sus métodos públicos lo serán) pero esto nos puede dar algún problema por lo que hay que ser cautos.

Algo que necesitamos tener claro es cuánto dura una transacción. Esto es, cuando empieza y cuando termina. Si tenemos anotado un método con `@Transactional`, la transacción empezará justo antes de la primera línea del método y terminará justo después de la última. Si dentro de este método existen llamadas a otros métodos estos otros métodos son llamados dentro de la transacción sin necesidad de anotarlos con `@Transactional`.

Ahora bien, ¿qué sucede cuando un método anotado con `@Transactional` llama a otro método anotado también con `@Transactional`?

```
public class A {
    @Autowired
    private B b;

    @Transactional
    public void hazAlgoA() {
        b.hazAlgoB();
    }
}

public class B {
    @Transactional
    public void hazAlgoB() {
        hazAlgoTransaccionalmente();
    }
}
```

¿Se mantiene la misma transacción? ¿Se abre una nueva? A esto se le llama **propagación de la transacción**.

Por defecto una transacción tiene una propagación de `PROPAGATION_REQUIRED`, si la transacción no existe la crea y si la tiene la aprovecha. Según esto, el primer método abre la transacción y el segundo la aprovecha por lo que todo está dentro de la misma transacción.

Ahora supongamos que queremos justo lo contrario. Necesitamos que el método de la `claseB` utilice una nueva transacción, de esta manera si algo falla en B, la

transacción de A no hace rollback. Tal como está actualmente no nos sirve por lo que hemos de modificar la propagación de B de la siguiente manera:

`@Transactional( propagation = Propagation.REQUIRES_NEW )`

```
public void hazAlgoB() {  
    hazAlgoTransaccionalmente();  
}
```

¡Ahora sí! al entrar en B, se generará una nueva transacción.

Spring soporta las siguientes propagaciones:

PROPAGATION_REQUIRED: Es la que viene programada por defecto, así que no es necesaria especificarla. Si existe transacción la aprovecha y sino la crea.

REQUIRES_NEW: Abre una transacción nueva y pone en suspenso la anterior. Una vez el método marcado como REQUIRES_NEW termina se vuelve a la transacción anterior.

PROPAGATION_SUPPORTS: Si existe transacción la aprovecha, sino no crea ninguna.

PROPAGATION_MANDATORY: Si no existe una transacción abierta se lanza una excepción. Hay programadores que anotan sus DAO con esta opción.

PROPAGATION_NEVER: Si existe una transacción abierta se lanza una excepción. No suele ser necesario utilizar esta propagación, pero existe.

PROPAGATION_NOT_SUPPORTED: Si existe una transacción la pone en suspenso, la transacción se reactiva al salir del método.

Para poder manejar bien las transacciones, debemos tener en cuenta que **todo método que pueda generar un cambio persistente en la base de datos, debe ser marcado como @Transactional**.

En los casos de los métodos de consulta, podemos encontrarnos que también están anotados como transaccionales, pero en estos casos debemos tener en cuenta su lectura.

La autorización se configura con la propiedad **readOnly** (solo lectura) de **@Transactional**. Su valor predeterminado es **false**, lo que significa que no está en modo de solo lectura, ergo se permite la escritura. Dicho con otras palabras, **@Transactional** siempre otorga permiso de escritura a la transacción, salvo que se indique lo contrario de manera explícita. La forma de declararlo es la siguiente:

```
@Transactional(readOnly = true)
```

Creando un servicio transaccional

Vamos a tomar como ejemplo la clase **CursoService** con la que ejemplificamos la clase repositorio. En esta clase vamos a anotar los métodos que realizan transacciones en la base de datos. Recordemos que en caso de que el método no vaya a realizar escrituras sobre la base de datos, podemos marcarlo como “solo lectura”, lo que le dará mayor beneficio de rendimiento. Cuando un método es anotado como **readOnly = true**, si se intenta realizar alguna operación de escritura sobre la BD, se generará una excepción.

```
@Service
```

```
public class CursoService{

    @Autowired

    private CursoRepository cursoRepository;
```

```
@Transactional(readOnly = true)
```

```
public List<Curso> listAllCursos() {

    return cursoRepository.findAll();

}
```

```
@Transactional
```

```
public Curso addCurso(Curso curso) {

    return cursoRepository.save(curso);

}
```

```
@Transactional
```

```
public int removeCurso(int id) {

    cursoRepository.delete(id);

    return 1;

}
```

```
@Transactional
```

```
public Curso updateCurso(Curso curso) {
```

```
return cursoRepository.save(curso);
```

```
}
```

```
}
```

Cierre 🖐️

En esta lección, aprendiste a gestionar, almacenar y recuperar datos relacionales en tu aplicación Java mediante el uso de JdbcTemplate y JPA.

Durante el recorrido, descubriste cómo configurar un DataSource en Spring y utilizar JdbcTemplate para llevar a cabo operaciones de consulta y modificación de datos. Además, conociste el mundo de JPA, donde las clases de entidad juegan un papel crucial al representar objetos que se almacenarán en la base de datos.

Una parte esencial de esta lección fue dar forma a la arquitectura de tus proyectos mediante la programación en capas. Aprendiste a invocar un repositorio desde un servicio para acceder a los datos de manera estructurada. También te familiarizaste con el uso de clases de repositorio, la realización de consultas parametrizadas y el manejo de la transaccionalidad en los servicios.

¡Nos vemos de nuevo, más adelante! 🖐️

Referencias

- Desarrollo Avanzado Web. (2013, 6 de octubre). Spring: Cómo mejorar JDBC. TodoSobreProgramacion. Recuperado de <http://todosobreprogramacion.blogspot.com/2013/10/conociendo-como-mejora-spring-jdbc.html>
- La Universidad de Alicante – Experto Java. (2015). Frameworks de persistencia – JPA. Universidad de Alicante. Recuperado de <https://expertojavaua.github.io/expertojava.ua.es/experto/restringido/2015-16/jpa/jpa.html> expertojavaua.github.io
- Peiretti, G. (s.f.). El componente Repository en Spring Boot. GustavoPeiretti.com. Recuperado de <https://gustavopeiretti.com/spring-boot-componente-repository/>
- Desarrollo Avanzado Web. (2013, 6 de octubre). Spring: Cómo manejar transacciones y sus estados. TodoSobreProgramacion. Recuperado de <https://todosobreprogramacion.blogspot.com/2013/10/spring-como-manejar-transacciones-y-sus.html>
- Danielme. (2023, 22 de febrero). Curso Spring Data JPA: transacciones, propagación y excepciones. Danielme.com. Recuperado de <https://danielme.com/2023/02/22/curso-spring-data-jpa-transacciones-propagacion-excepciones/>

¡Muchas gracias!

Nos vemos en la próxima lección

